06-docker-realtime.md

🚨 WHAT'S WRONG (FIXED VERSION)

❌ "Volume is simply a directory inside container"

👉 Not correct

✅ **Correct:**

- Volume = **storage managed by Docker**
- Stored on **host**, not inside container filesystem

❌ "When we create a container volume will be created"

👉 Wrong

✅ **Correct:**

- Volume is created **only if you specify it**
- Otherwise → container uses temporary filesystem (deleted later)

❌ "You can declare volume only while creating container"

👉 Wrong

✅ **Correct:**

- You can create volume anytime:

docker volume create mydata

❌ "We can't create volume from existing container"

👉 Wrong / half knowledge

✅ **Correct:**

- You can create and attach volumes later using:
  - `docker run`
  - `docker-compose`
- But you can't magically extract existing container filesystem into volume without manual steps

❌ “Volume is inside container”

👉 This is the biggest misunderstanding

👉 Reality:

Host Machine
↓
/var/lib/docker/volumes/
↓
Container mounts it

👉 Container just **uses it**, doesn’t own it

✅ CORRECT MENTAL MODEL (THIS IS WHAT YOU REMEMBER)

🧠 Think like this:

Thing	Meaning
Container	Temporary worker
Volume	External hard disk
Image	Blueprint

📦 REAL-WORLD EXAMPLE

Without volume:

```
docker run -it ubuntu
touch file.txt
exit
docker rm <container>
```

👉 file.txt → ❌ gone

With volume:

```
docker run -it -v mydata:/data ubuntu
cd /data
touch file.txt
```

👉 delete container
👉 run again:

docker run -it -v mydata:/data ubuntu

👉 file.txt → ✅ still there

🔁 TYPES OF VOLUME MAPPING

1️⃣ Container ↔ Container

docker run -v mydata:/data container1
docker run -v mydata:/data container2

👉 shared storage

2️⃣ Host ↔ Container (VERY IMPORTANT)

docker run -v /home/harish/data:/data ubuntu

👉 `/home/harish/data` = real folder on server
👉 `/data` = inside container

⚠️ THINGS YOU MISSED (IMPORTANT)

❗ Volumes are NOT part of image

👉 When you do:

docker build

👉 volumes are NOT included

❗ Containers are ephemeral

👉 If you store DB data inside container → you will lose it

👉 Always use volume for:

- DB
- logs
- uploads

🔥 YOUR CURRENT GAP

You’re thinking:

> container = server

👉 Wrong.

You should think:

> container = process
> volume = storage

🎯 FINAL SIMPLE MEMORY

👉 **Container dies → Volume survives**

🚀 NEXT STEP (YOU SHOULD DO)

Try this:

docker volume create testvol

docker run -it -v testvol:/data ubuntu

👉 create file → delete container → run again
👉 see persistence

short cut notes

why volumes: to replicate the data into other containers we use volume
Volume = Shared folder on your host machine
2 ways to create 1. dockerfile 2. command pass
1 st way
--privileged=true --volumes-from cont1 & --volumes-from ← THIS is what accesses cont1's volumes to cont2
--volumes-from cont1:
- Copies ALL volume mounts from cont1
- new-container2 sees same data as cont1
- YES, data comes from cont1's volumes ✅

cont2 updates data → cont1 sees it immediately!Shared storage ✅ (both access same location)

docker run -it --name cont1 -v /FLM-Volume ubuntu
docker run -it --name cont2 --privileged=true --volumes-from harish1 ubuntu

or
docker run -it --name cont1 -v FLM-Volume:/app/data ubuntu
docker run -it --name cont2 --volumes-from cont1 ubuntu

Option A: Host path to container
docker run -it --name cont1 -v /FLM-Volume:/app/data ubuntu

Option B: Named volume
docker run -it --name cont1 -v FLM-Volume:/app/data ubuntu

Option C: Named volume (simpler)
docker volume create FLM-Volume
docker run -it --name cont1 -v FLM-Volume:/data ubuntu

2.
image will not update when you update image
inside cont volume create 3 files
inside cont created 10 files
exit --> cont2 create image

docker commit cont2 my_image
Now from this image i will create container i.e
docker run -it --name cont3 my_image
after enter into the container3 only container files you will see not volume
when you go to volume folder it is empty
volume will not update when you update an image
Next point is even if we stop container or remove we can still access the volume

Container 1:
├── Volume: 3 files (in /app/data)
└── Filesystem: 10 files (in container)

docker commit cont2 my_image
↓
Image includes: 10 files ✅
Image includes: 3 volume files? ❌ (volumes NOT in image!)

Container 3 (from my_image):
├── Filesystem: 10 files ✅ (from image)
├── Volume: EMPTY ❌ (not in image)
└── Volume folder: Empty unless mounted ✅

Key Point:
Images ≠ Volumes
Volumes are independent!

data sharing till now happen container to container

3.
Key Difference

Feature	Named Volume	Bind Mount
Where stored	Docker manages	You choose (/root/myapp)
Control	Docker	You (full control)
Sync	Automatic	Automatic
Visibility	Hidden in Docker folder	Your folder
Best for	Production	Development

Named Volume: -v FLM-Volume:/data
- Docker manages location
- Hidden from you
- Production use

Bind Mount: -v /root/myapp:/data
- YOU control location
- Visible in your folder
- Development use
- Changes sync instantly ✅

host to container how to replicate host : means where you run data
If you update the folder attached containers eill update automatically will see
mkdir myapp
this folder make as a host and how to give
docker run -it --name cont-one -v /root/myapp:/Mustafa-volume --privileged=true ubuntu bash
myapp is host
mustafa-volume is volume name
inside cont volume created 3 files aws azure gcp
then i exit and cam back to myapp folder then here also iam able to see 3 files
out side host file i put content ex: vim aws hey bro iam harish again enter container and check cat aws you will see hey bro iam harish
container also data is updated
cont1 lo unna same data cont2 ki vellali same command

docker run -it --name cont-two -v /root/myapp:/harish-volume --privileged=true ubuntu bash
here container 2 volume also come same files

we have alternative command also
docker run -it --name cont-two -v $(pwd):/harish-volume --privileged=true ubuntu bash

docker volume create harish (to create own volume without container)
while create container i assign volume
Now separate create volume one i will keeep all files and new created container can we keep : yes

docker volume create paytm
cd /var/lib/docker/volumes/paytm/_data
touch paytm-files{1..5}
docker run -itd --name cont99 --mount source=paytm,destination=/my-volume ubuntu
so here my-volume is mounted
cd my-volume
we can see the files as we created

docker volume create zomato
cd volumes/zomoto/_data
vim index.html
<h1> hey this is harish</h>
docker run -it --name zomato --mount source=zomato,destination=/usr/share/nginx/html -p 8089:80 nginx
copy ip adresspaste with url

-v also we can do command
docker run -itd --name cont-1 -v /usr/share/nginx/html -p 8082:80 nginx
-v volume declare where means /usr/share/nginx/html
so e path lo create avatadi

git clone https://github.com/Harishjangidi/staticsite-docker.git
cd html
docker run -itd --name cont-2 -v $(pwd) /usr/local/apache2/htdocs -p 8083:80 httpd

Docker file volume can create

🧠 ONE LINE DEFINITION (remember this)

👉 **Volume = a place to store data outside the container so it doesn't get lost**

🏠 Real-world analogy (5th class level)

Think like this:

🧳 Container = Temporary Room

- You stay in hotel → room gets cleaned after you leave
- Data inside container = temporary ❌

📦 Volume = Locker outside room

- You keep important things in locker
- Even if room changes → your data is safe ✅

🔁 WHY WE USE VOLUMES

❌ Without volume

docker run ubuntu

- create files → exit → gone ❌

✅ With volume

docker run -v mydata:/data ubuntu

- create files → exit → still there ✅

🔗 3 USE CASES (THIS IS YOUR CORE STORY)

1️⃣ Persist data (MOST IMPORTANT)

Real world:

Database → data must not disappear

Syntax:

docker run -v mydata:/data ubuntu

👉 Restart container → data still there

2️⃣ Share data between containers

Real world:

2 apps using same database

Syntax:

docker run -it --name cont1 -v mydata:/data ubuntu
docker run -it --name cont2 -v mydata:/data ubuntu

👉 Both see same files ✅

⚠️ Your mistake:
You used:

--volumes-from

👉 This is **old + rarely used**
👉 Better use **same named volume**

3️⃣ Connect host ↔ container (MOST PRACTICAL)

Real world:

You edit code → container instantly uses it

Syntax:

docker run -v /root/myapp:/app ubuntu

👉 Host = `/root/myapp`
👉 Container = `/app`

🔥 KEY TYPES (don't confuse)

🟢 Named Volume

-v mydata:/data

- Docker manages
- hidden
- production use

🔵 Bind Mount

-v /root/myapp:/data

- YOU control
- visible
- development use

❌ BIG CONFUSION YOU HAD (IMPORTANT)

Images vs Volumes

Your statement:

> volume will not update when you update image

👉 Correct — but understand WHY:

🧠 Concept

Image = blueprint 📄

Container = running machine ⚙️

Volume = external storage 📦

Example flow:

docker run -v mydata:/data ubuntu

Inside container:

- `/data` → stored in volume
- other files → inside container

Now:

docker commit cont1 myimage

👉 What gets saved?

- container files ✅
- volume data ❌

New container:

docker run myimage

👉 volume is EMPTY ❌
👉 because volume is separate

🔁 SIMPLE MEMORY TRICK

👉 **3 layers**

[IMAGE] → code only
[CONTAINER] → runtime
[VOLUME] → data storage

👉 Volume is NEVER inside image

⚠️ About --privileged

You're overusing this.

👉 It has NOTHING to do with volumes

--privileged

= full system access (danger)

❌ Not required for:

- volume
- sharing data
- development

🔥 CLEAN COMMANDS YOU SHOULD PRESENT

Create volume

docker volume create mydata

Use volume

docker run -it -v mydata:/data ubuntu bash

Bind mount

docker run -it -v /root/myapp:/app ubuntu bash

Share between containers

docker run -it -v mydata:/data ubuntu
docker run -it -v mydata:/data ubuntu

🧨 What you should REMOVE from your explanation

- ❌ `--privileged=true` (not related)
- ❌ overuse of `--volumes-from`
- ❌ mixing image + volume logic

🎯 Final simple explanation (use this in presentation)

👉 *“Container is temporary like a hotel room. If we store data inside it, it will be lost when container is deleted. So we use volumes, which act like a locker outside the container. This ensures data is safe, can be shared between containers, and even connected to the host system.”*
